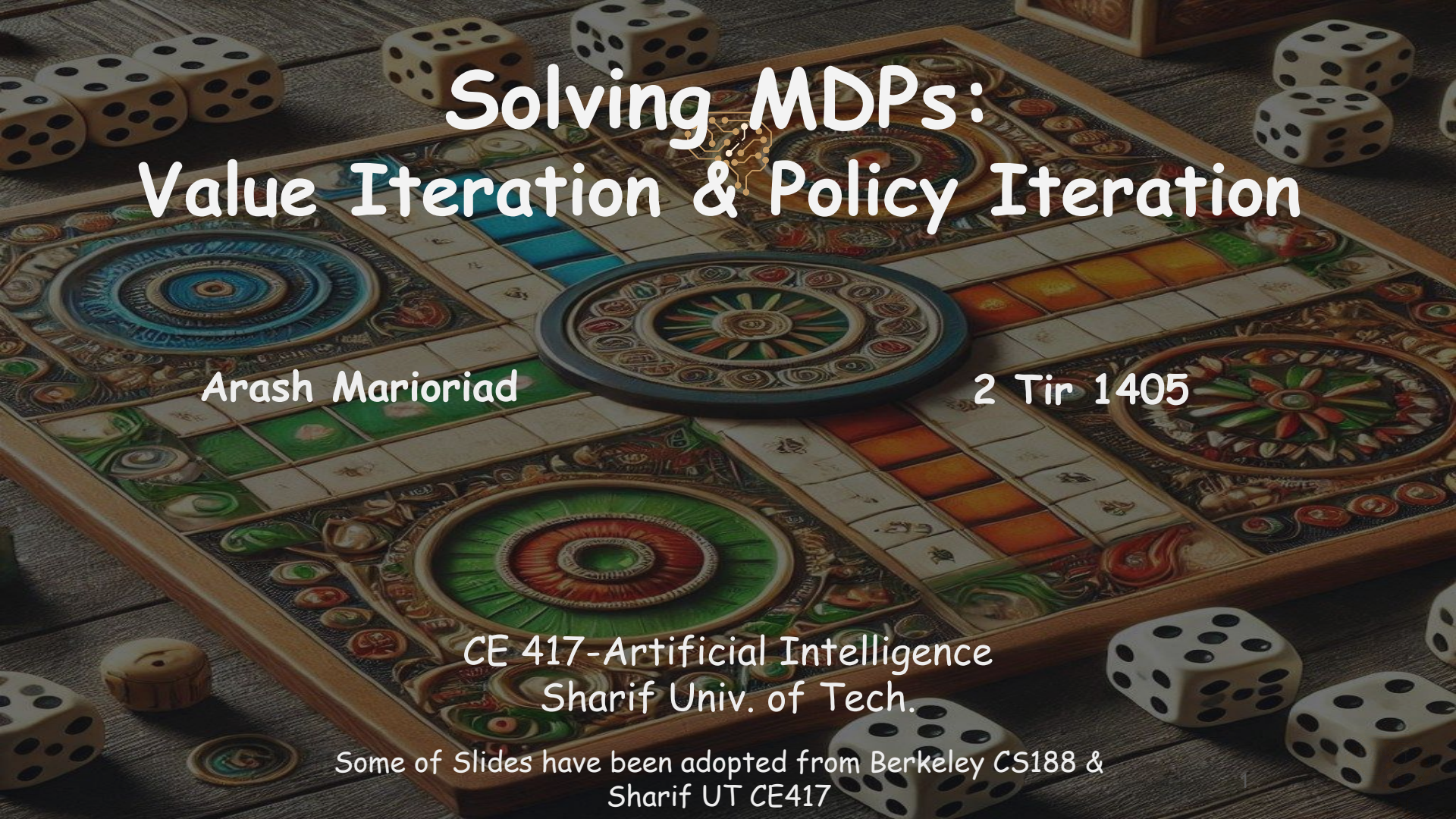


Solving MDPs: Value Iteration & Policy Iteration

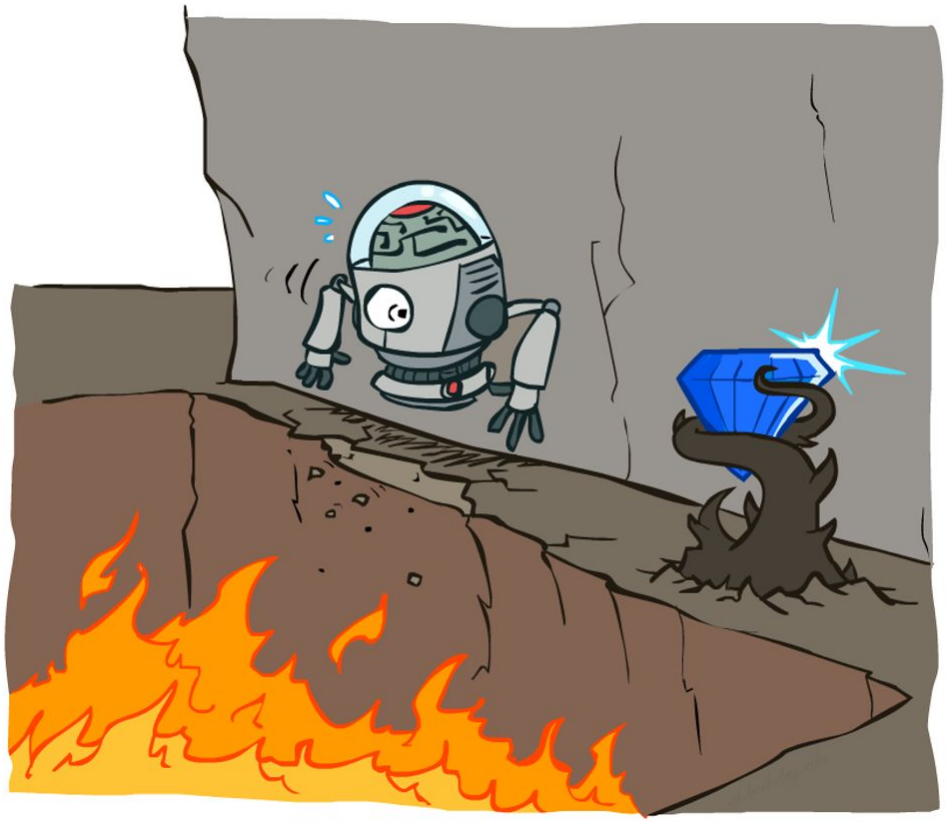
Arash Marioriad

2 Tir 1405

CE 417-Artificial Intelligence
Sharif Univ. of Tech.

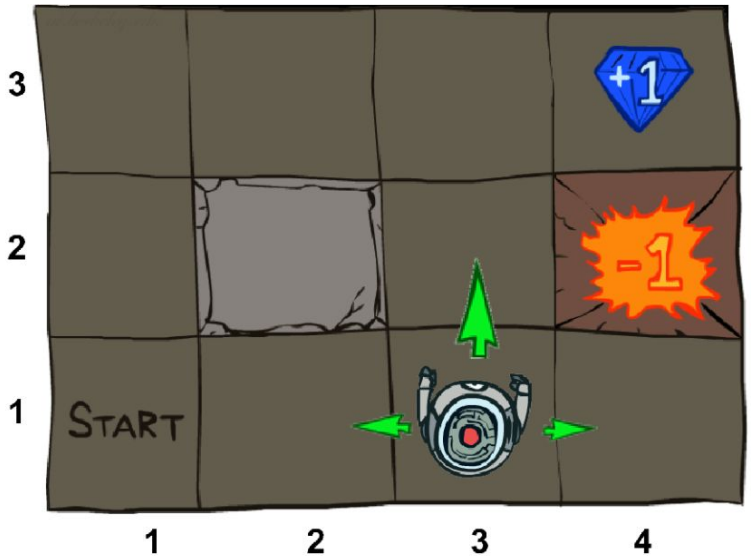
Some of Slides have been adopted from Berkeley CS188 & Sharif UT CE417

Non Deterministic Search

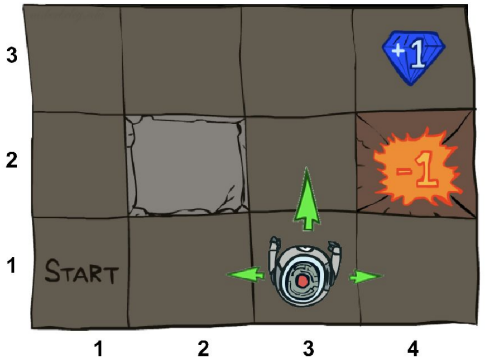


Example: Grid World

- A maze-like problem
  - The agent lives in a grid
  - Walls block the agent's path
- Noisy movement: actions do not always go as planned
  - 80% of the time, the action North takes the agent North (if there is no wall there)
  - 10% of the time, North takes the agent West; 10% East
  - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
  - Small "living" reward each step (can be negative)
  - Big rewards come at the end (good or bad)
- Goal: maximize sum of rewards



Markov Decision Processes (MDP)



- An MDP is defined by:
  - A set of states $s \in S$
  - A set of actions $a \in A$
  - A transition function $T(s, a, s')$
    - Probability that a from s leads to s', i.e., $P(s' | s, a)$
    - Also called the model or the dynamics
  - A reward function $R(s, a, s')$
    - Sometimes just $R(s)$ or $R(s')$
  - A start state
  - Maybe a terminal state

Formulation: What is Markov about MDPs?

- "Markov" generally means that given the present state, the future and the past are independent
- For Markov decision processes, "Markov" means action outcomes depend only on the current state

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \ldots S_0 = s_0)$$

$$=$$

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

- This is just like search, where the successor function could only depend on the current state (not the history)

Formulation: What is the purpose of the agent?

maximize it’s reward across all timesteps **(Return/Utility)**:

$$U([s_0, a_0, s_1, a_1, s_2, \ldots]) = R(s_0, a_0, s_1) + R(s_1, a_1, s_2) + R(s_2, a_2, s_3) + \ldots$$

Discounting

- It’s reasonable to maximize the sum of rewards
- It’s also reasonable to prefer rewards now to rewards later
- One solution: values of rewards decay exponentially

1

Worth Now

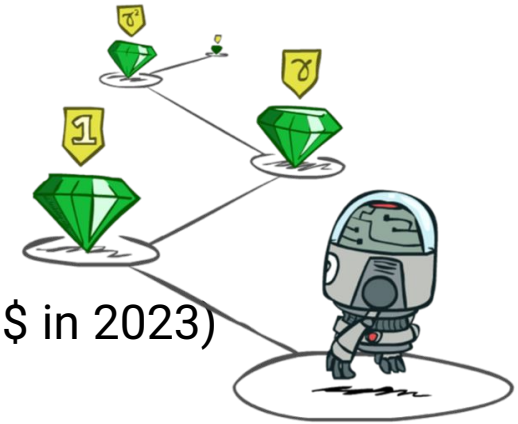
γ

Worth Next Step

γ^2

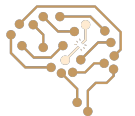
Worth In Two Steps

Finite Horizons and Discounting



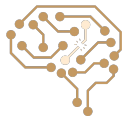
- Problems with Utility Function:
  - obtaining infinite reward?
  - prefer rewards now to rewards later (1$ in 2000 > 1$ in 2023)
- Solutions:
  - finite horizons: defines a "lifetime" for agents
  - discount factors: additive utility -> discounted utility (get rid of infinite utility)

$$U([s_0, a_0, s_1, a_1, s_2, \ldots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \ldots$$

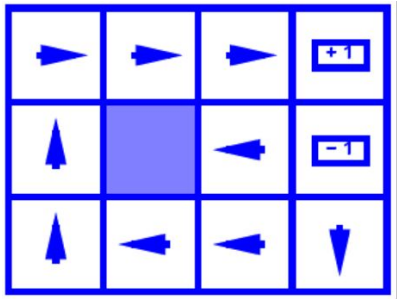


Optimal Policy

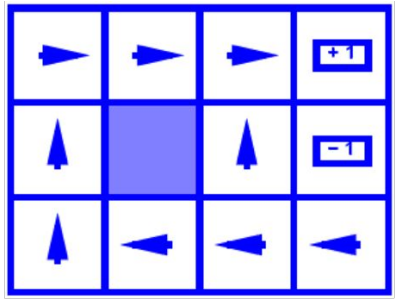
- For MDPs, we want an optimal policy $\pi^*: S \rightarrow A$
  - A policy π gives an action for each state
  - An optimal policy is one that **maximizes expected utility** if followed
- Expectimax didn't compute entire policies
  - It computed the action for a single state only



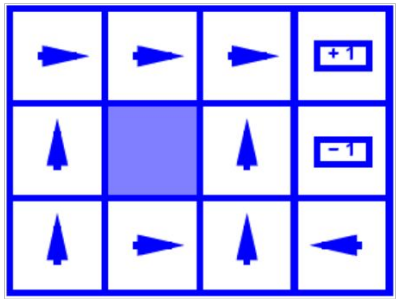
Gridworld Optimal Policy Example



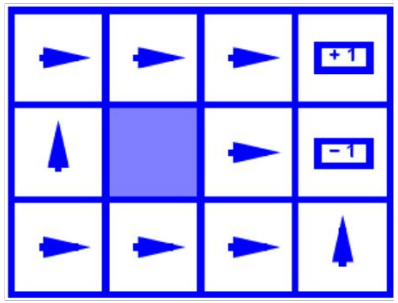
R(s) = -0.01



R(s) = -0.03



R(s) = -0.4



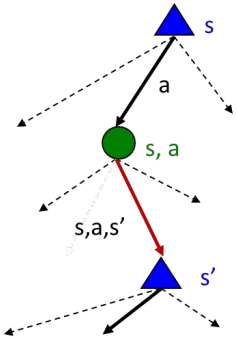
R(s) = -2.0

How To Find Optimal Policy in MDP?

- Two Iterative Algorithms:
  - **Value Iteration**
  - **Policy Iteration**

- First, let’s define some notations …

Optimal Quantities



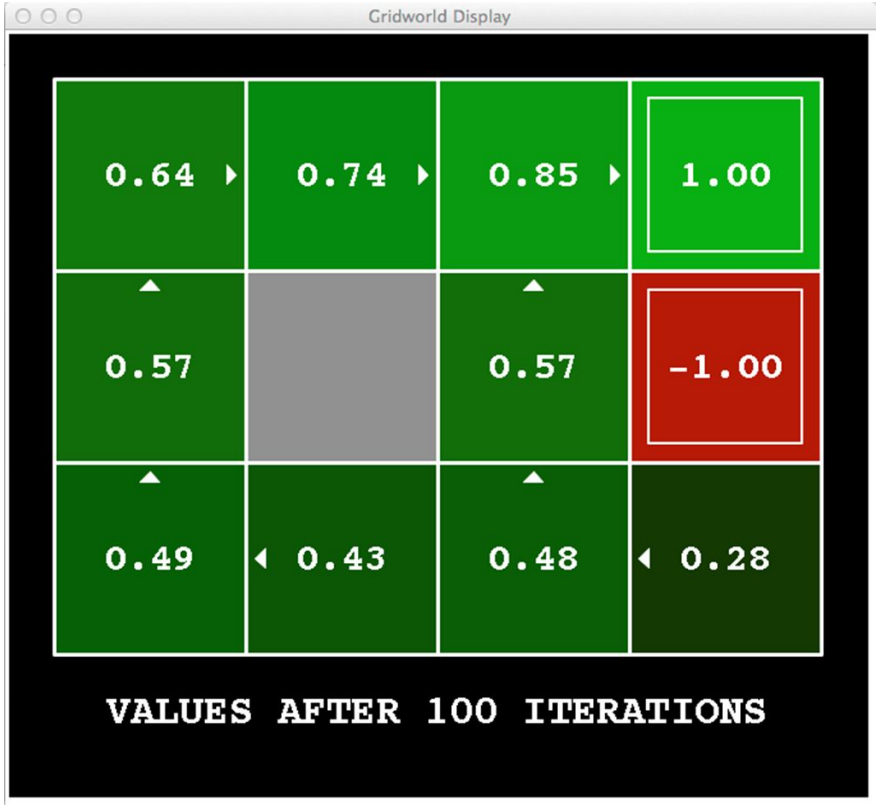
s is a *state*

(s, a) is a *q-state*

(s,a,s') is a *transition*

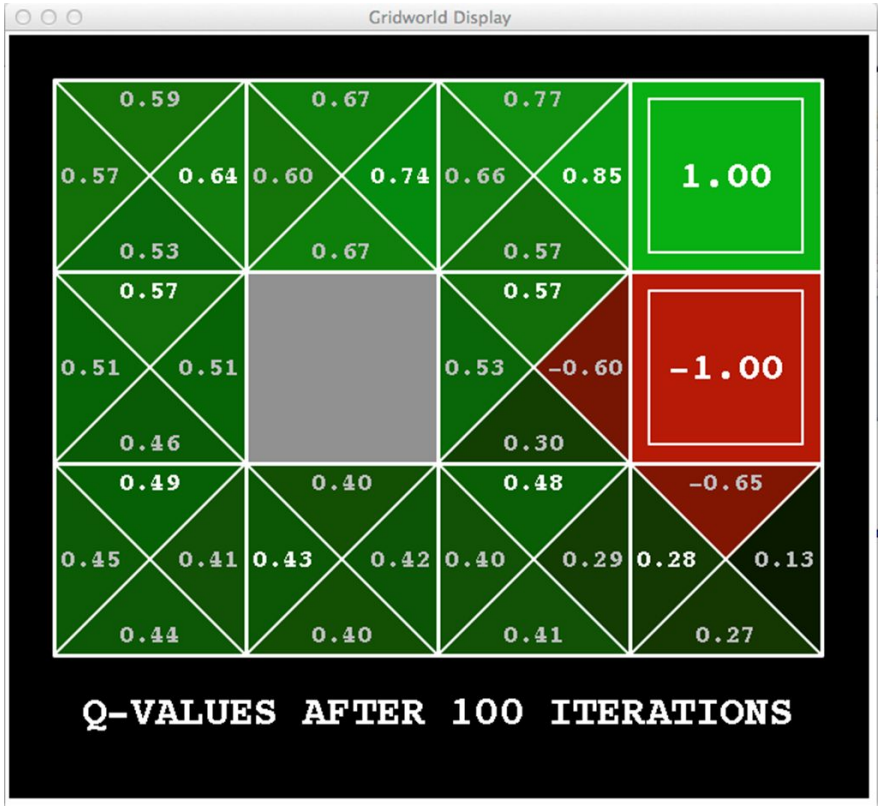
- The optimal value of a state s:
  - **V*(s)** = expected utility starting in s and acting optimally
- The optimal value of a state-action pair (s,a):
  - **Q*(s,a)** = expected utility starting out having taken action a from state s and (thereafter) acting optimally
- The optimal policy:
  - **π*(s)** = shows optimal action for state s

Example: Optimal Values



Noise = 0.2
Discount = 0.9
Living reward = 0

Example: Optimal Q-Values



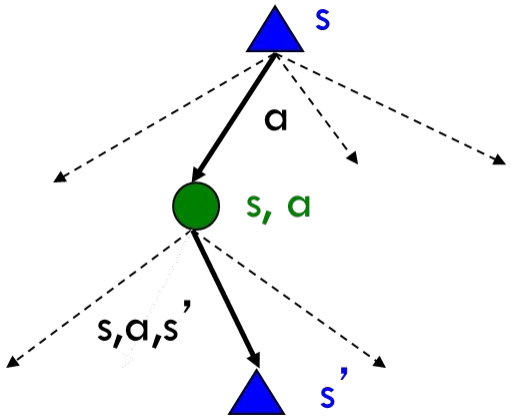
Noise = 0.2
Discount = 0.9
Living reward = 0

The Bellman Optimality Equations

$$V^*(s) = \max_a Q^*(s,a)$$

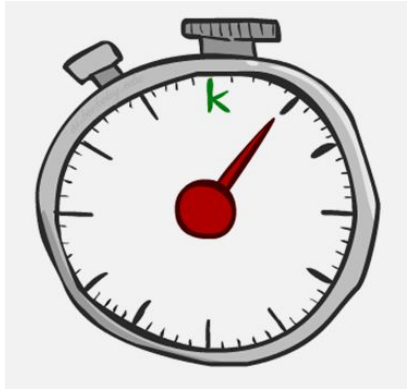
$$Q^*(s,a) = \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s') \right]$$

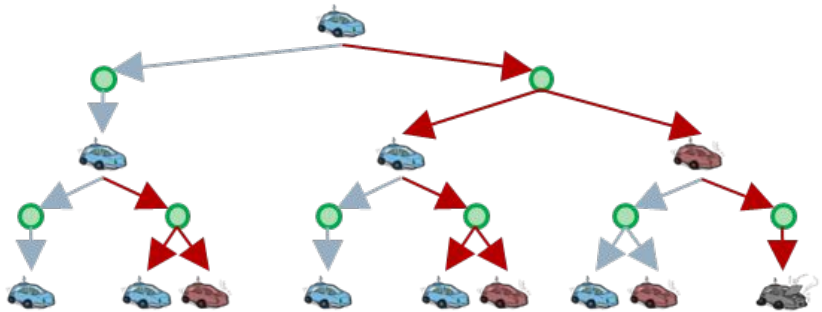
$$V^*(s) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s') \right]$$

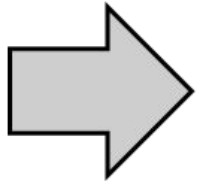


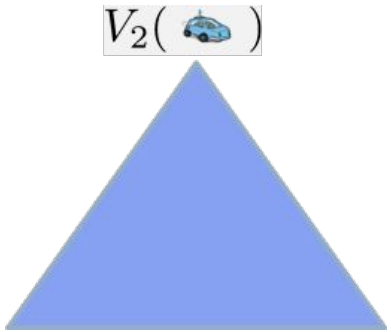
Time-Limited Values

- Define $\mathbf{V_k(s)}$ to be the optimal value of s if the game ends in k more time steps

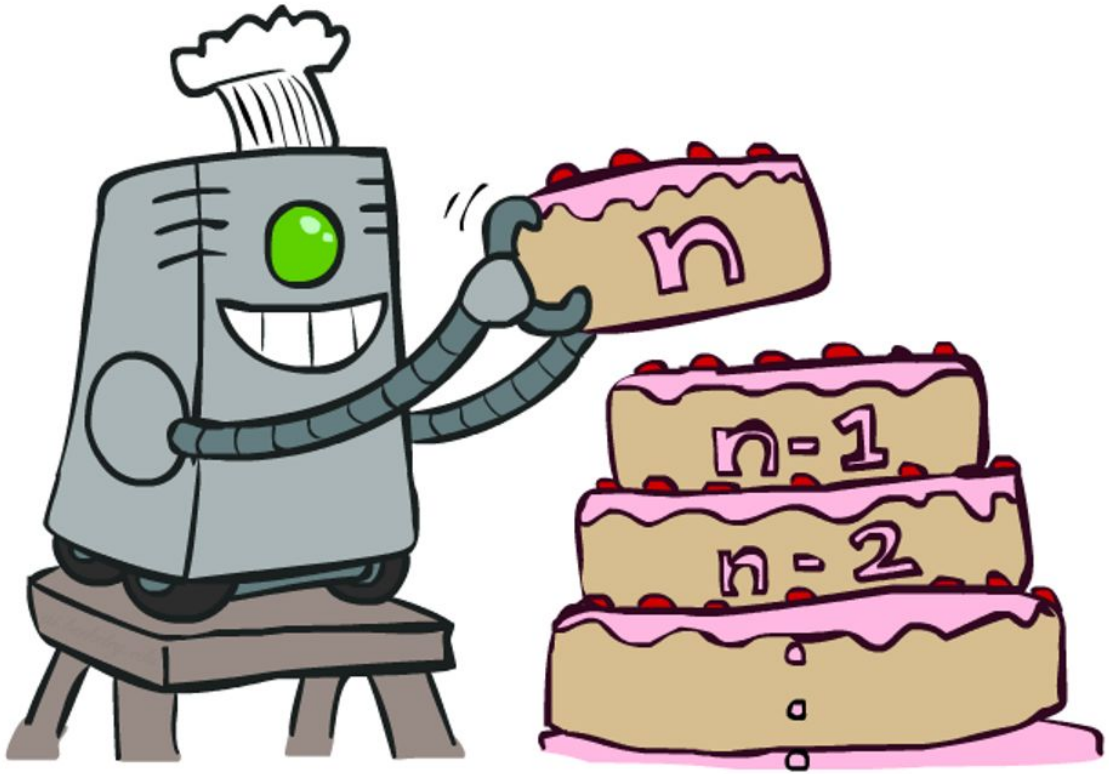








Value Iteration Algorithm



Value Iteration

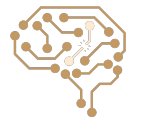
- Bellman equations characterize the optimal values:

$$V^*(s) = \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s') \right]$$

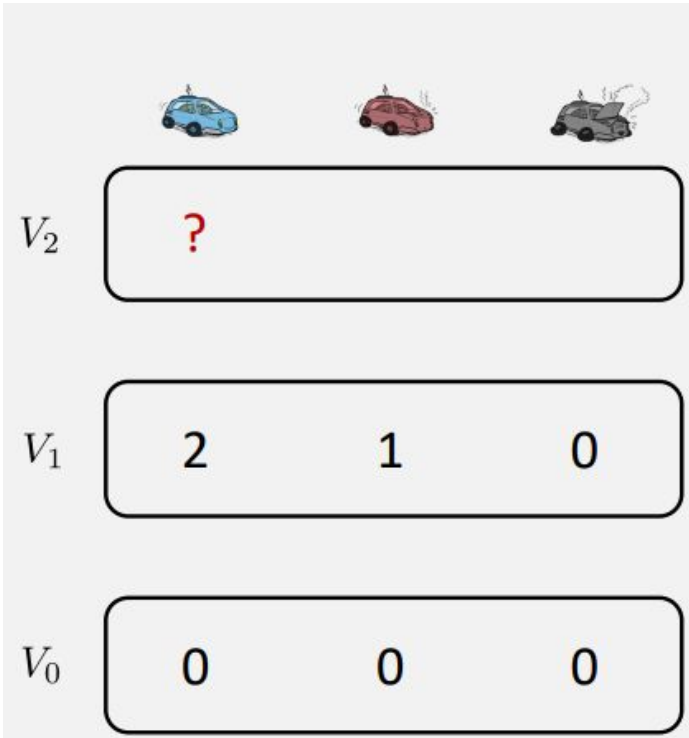
- Start with $V_0(s) = 0$: no time steps left means an expected reward sum of zero
- Repeat until convergence

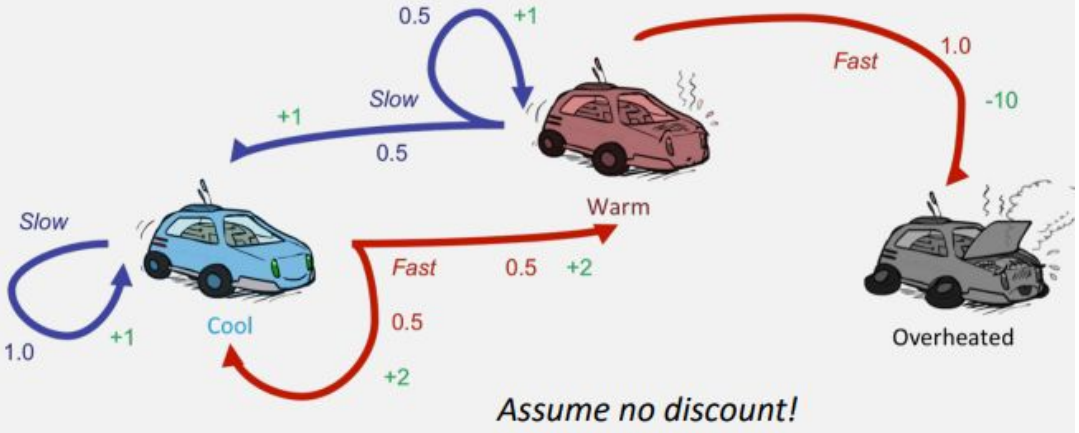
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- Complexity of each iteration: $O(S^2A)$



Example: Value Iteration





Assume no discount!

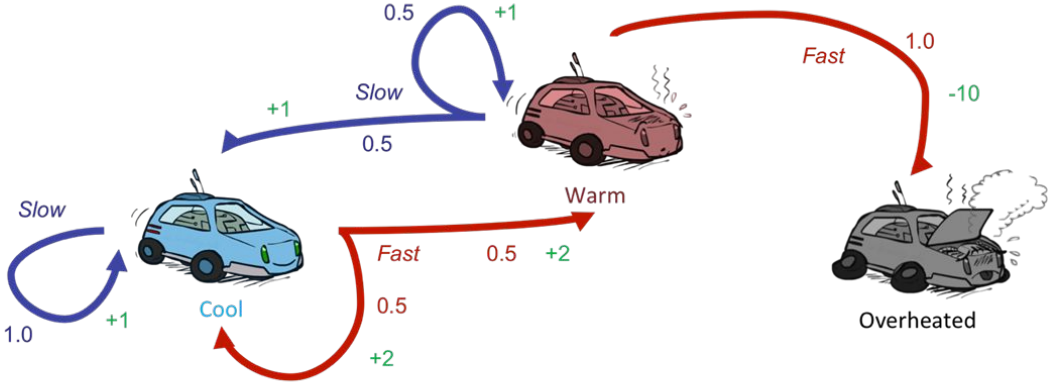
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

a=slow: $1(1 + 2) = 3$

a=fast: $0.5(2 + 2) + 0.5(2 + 1) = 3.5$

Example: Value Iteration

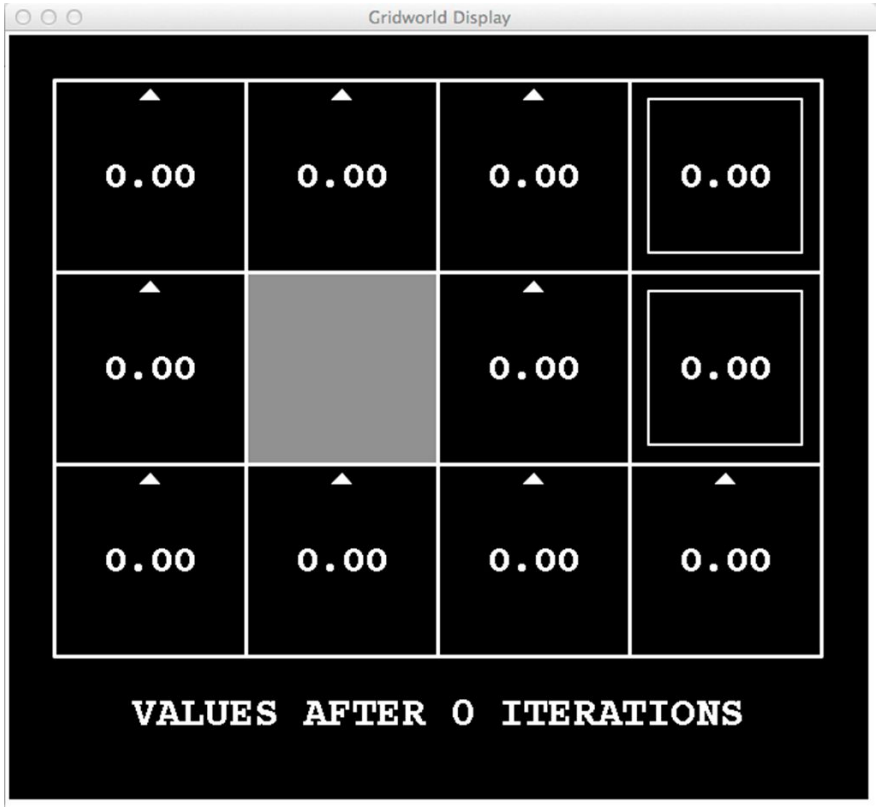
V_2	3.5	2.5	0
V_1	2	1	0
V_0	0	0	0



Assume no discount!

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

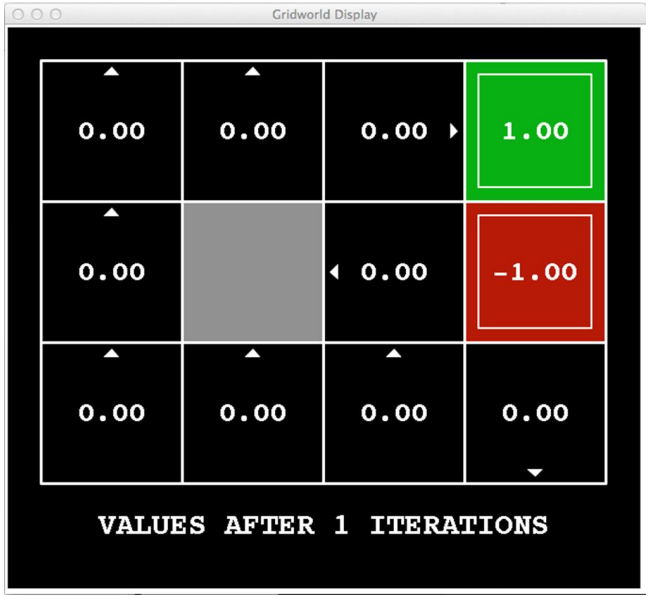
Value Iteration: k=0



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=1

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=2

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=3

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

Noise = 0.2
Discount = 0.9
Living reward = 0

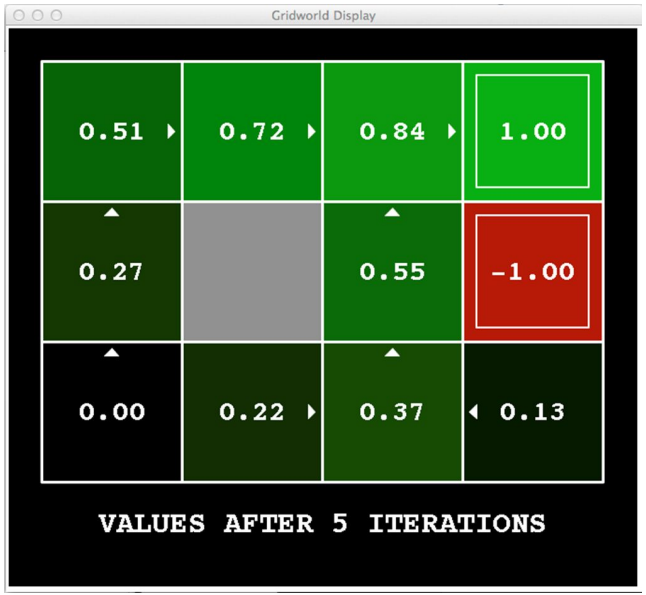
Value Iteration: k=4

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=5

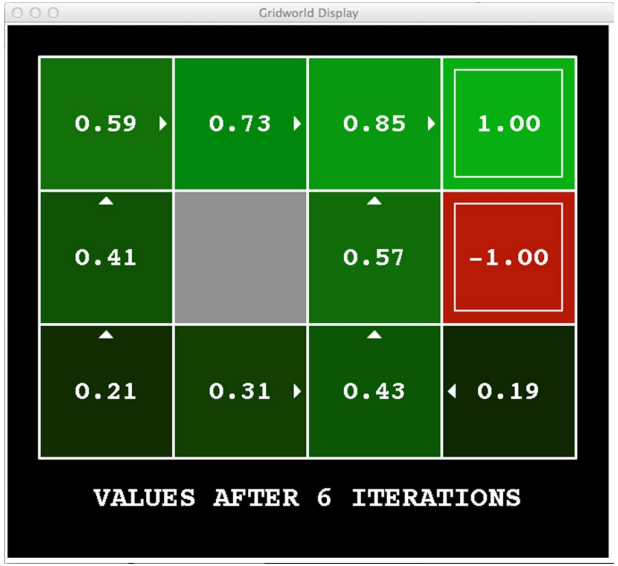
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=6

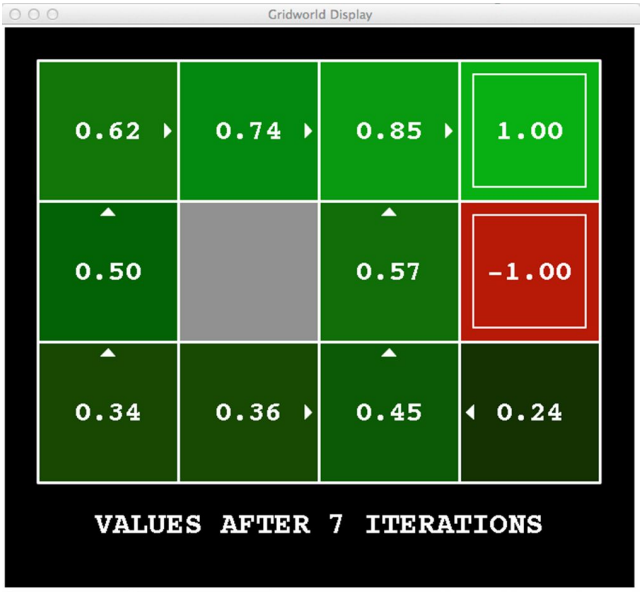
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=7

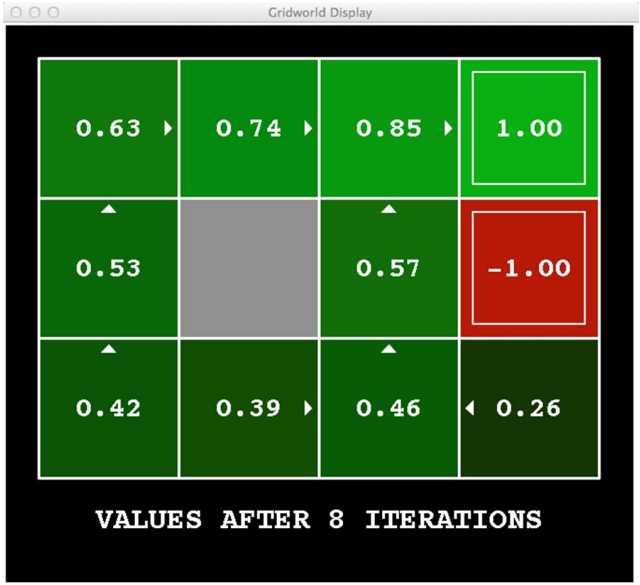
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=8

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$



Noise = 0.2
Discount = 0.9
Living reward = 0

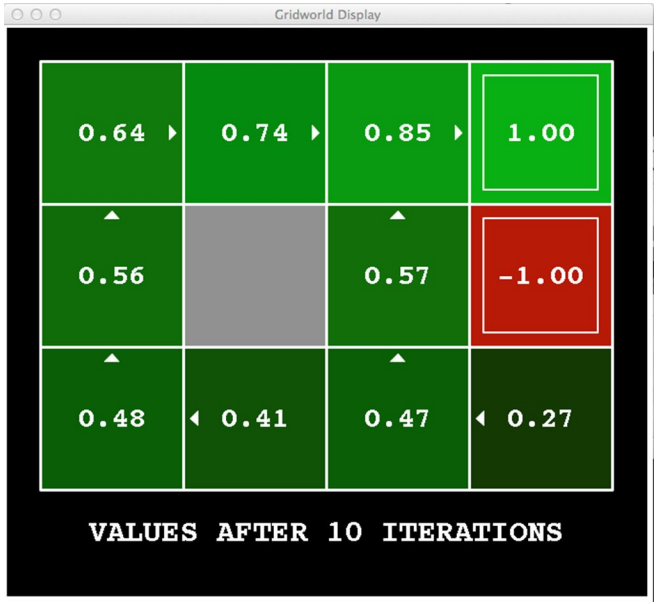
Value Iteration: k=9

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=10

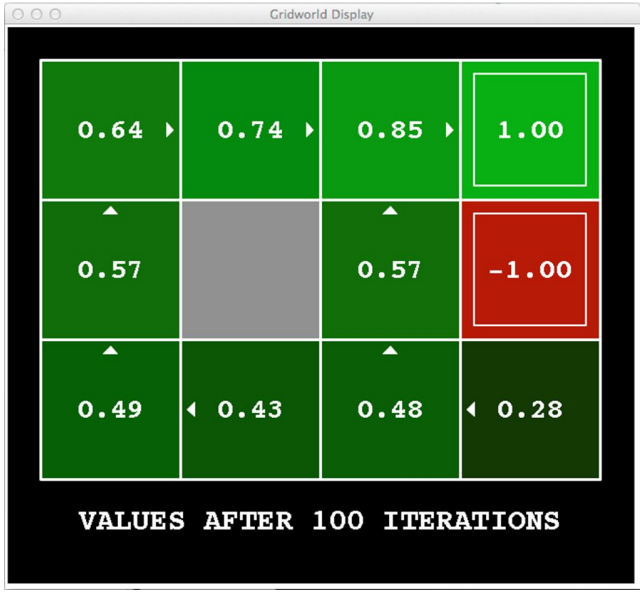
$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$



Noise = 0.2
Discount = 0.9
Living reward = 0

Value Iteration: k=100

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s,a)(R(s,a,s') + \gamma V_k(s'))$$

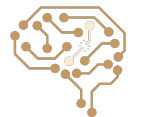


Noise = 0.2
Discount = 0.9
Living reward = 0

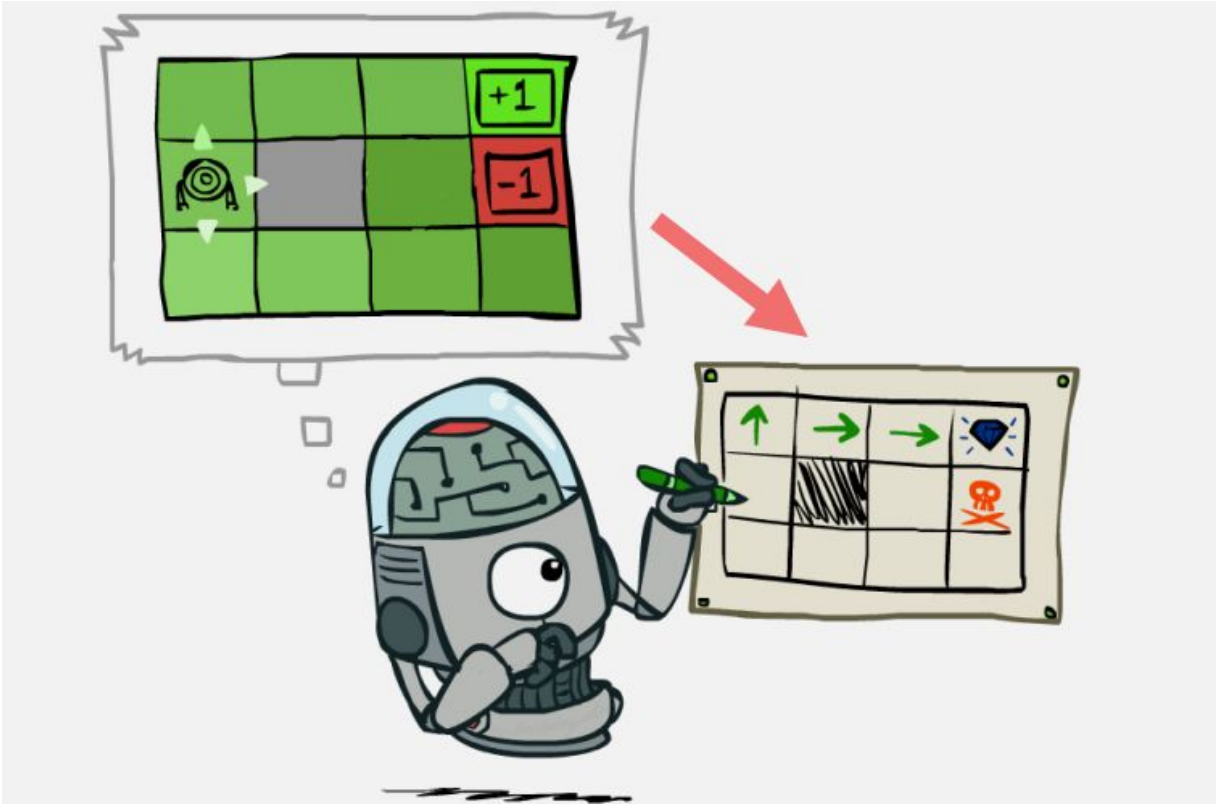
Value Iteration Convergence

Theorem. Value iteration converges. At convergence, we have found the optimal value function V* for the discounted infinite horizon problem, which satisfies the Bellman equations

$$\forall S \in S: \quad V^*(s) = \max_A \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V^*(s') \right]$$

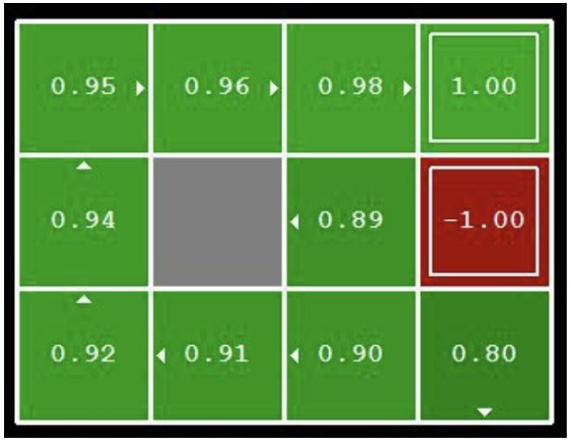


Policy Extraction



Policy Extraction: Computing Actions from Values

- Let's imagine we have the optimal values V*(s)
- How should we act?
  - It's not obvious!
- We need to do a mini-expectimax (one step)

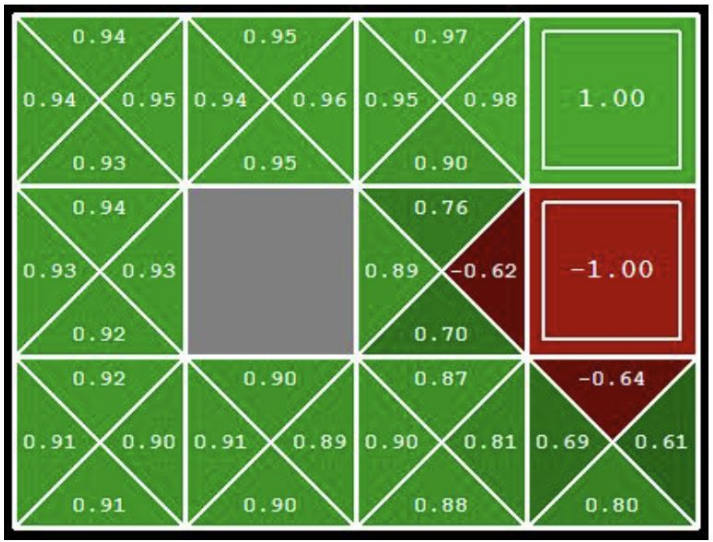


$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

- This is called policy extraction, since it gets the policy implied by the values

Policy Extraction: Computing Actions from Q-Values

- Let's imagine we have the optimal q-values:



- How should we act?
  - Completely trivial to decide!

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

- Important lesson: actions are easier to extract from q-values than values!

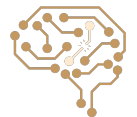
Algorithm 2: Value Iteration Algorithm

Data: θ: a small number

Result: π: a deterministic policy s.t. $\pi \approx \pi_*$

Function *ValueIteration* **is**

```
    /* Initialization                                          */
    Initialize V(s) arbitrarily, except V(terminal);
    V(terminal) ← 0;
    /* Loop until convergence                                  */
    Δ ← 0;
    while Δ < θ do
        for each s ∈ S do
            v ← V(s);
            V(s) ← max_a Σ_{s',r'} p(s', r|s, a)[r + γV(s')];
            Δ ← max(Δ, |v − V(s)|);
        end
    end
    /* Return optimal policy                                   */
    return π s.t. π(s) = arg max_a Σ_{s',r'} p(s', r|s, a)[r + γV(s')];
end
```



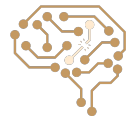
Problems with Value Iteration

- Value iteration repeats the Bellman updates:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s') \left[R(s,a,s') + \gamma V_k(s') \right]$$

- Problem 1: It's slow – $O(S^2A)$ per iteration
- Problem 2: The "max" at each state rarely changes
- Problem 3: The policy often converges long before the values

Value Iteration: k=12



Value Iteration: k=100

Next Session: Policy Iteration Algorithm